

# Lesson 5 — Program to Add Two Sparse Matrices

## 1. Where We Are Now

You have already learned the foundation needed for sparse matrix addition.

### Lesson 1: Sparse Matrix Foundations

A sparse matrix is a matrix with mostly zero values.

Instead of storing every value, we store only the values that are not zero.

### Lesson 2: Coordinate List Representation

Each non-zero element is stored using three pieces of information:

```
row, column, value
```

Example:

```
(2, 3, 7)
```

means:

```
At row 2, column 3, the value is 7.
```

### Lesson 3: Displaying a Sparse Matrix

You learned how to display the full matrix even though only the non-zero elements are stored.

The program visits every position in the full matrix.

If the position exists in the sparse list, it prints the stored value.

If the position does not exist in the sparse list, it prints `0`.

### Lesson 4: Sparse Matrix Addition Algorithm

You learned the logic for adding two sparse matrices.

The important idea was:

Sparse matrix addition is like merging two sorted lists.

The algorithm uses three index variables:

```
i = current element in the first matrix  
j = current element in the second matrix  
k = current position in the result matrix
```

## Lesson 5: Current Lesson

Lesson 5 turns the addition algorithm into an actual program function.

In this lesson, we write the function that adds two sparse matrices and returns a new sparse matrix as the result.

---

## 2. Main Goal of Lesson 5

The goal of Lesson 5 is to write this function:

```
struct Sparse* add(struct Sparse *S1, struct Sparse *S2)
```

This function receives two sparse matrices:

```
S1 = first sparse matrix  
S2 = second sparse matrix
```

It returns a pointer to a new sparse matrix:

```
sum = S1 + S2
```

So the function does this:

```
Take two sparse matrices.  
Check if they can be added.  
Create a new result matrix.  
Merge the two sparse lists.  
Return the result matrix.
```

---

### 3. Data Structures Used

We use the same two structures from earlier lessons.

#### 3.1 Element Structure

One `Element` stores one non-zero value.

```
struct Element {  
    int i; // row index  
    int j; // column index  
    int x; // value  
};
```

#### Meaning of the fields

| Field          | Meaning                             |
|----------------|-------------------------------------|
| <code>i</code> | Row index                           |
| <code>j</code> | Column index                        |
| <code>x</code> | Value stored at that row and column |

Example:

```
Element e;  
e.i = 2;  
e.j = 5;  
e.x = 9;
```

This means:

At row 2, column 5, the value is 9.

Or simply:

```
Matrix[2][5] = 9
```

## 4. Sparse Structure

The `Sparse` structure stores information about the whole sparse matrix.

```
struct Sparse {  
    int m;           // number of rows  
    int n;           // number of columns  
    int num;          // number of non-zero elements  
    Element *e;      // dynamic array of non-zero elements  
};
```

### Meaning of the fields

| Field            | Meaning                                           |
|------------------|---------------------------------------------------|
| <code>m</code>   | Total number of rows                              |
| <code>n</code>   | Total number of columns                           |
| <code>num</code> | Number of non-zero elements                       |
| <code>e</code>   | Pointer to the dynamic array of non-zero elements |

Example:

```
m = 3  
n = 4  
num = 2
```

This means:

```
The full matrix has 3 rows.  
The full matrix has 4 columns.  
Only 2 values are non-zero.
```

Important:

```
num does not mean m * n.
```

`num` means only the number of stored non-zero elements.

## 5. What the add() Function Should Do

The `add()` function should perform these steps:

1. Check if the matrix dimensions match.
2. Create a new result matrix on the heap.
3. Copy the row and column size into the result matrix.
4. Allocate enough memory for the result elements.
5. Use i, j, and k to merge the two sparse lists.
6. Add values when two elements have the same row and column.
7. Copy leftover elements after one list ends.
8. Set the final number of non-zero elements.
9. Return the result matrix.

---

## 6. Function Header

The function header is:

```
struct Sparse* add(struct Sparse *S1, struct Sparse *S2)
```

### What this means

|                                |                                                       |
|--------------------------------|-------------------------------------------------------|
| <code>struct Sparse*</code>    | The function returns a pointer to a Sparse structure. |
| <code>add</code>               | The function name is add.                             |
| <code>struct Sparse *S1</code> | The first input matrix is passed by pointer.          |
| <code>struct Sparse *S2</code> | The second input matrix is passed by pointer.         |

### Why do we pass pointers?

We pass pointers because sparse matrices contain dynamic arrays.

Passing pointers is efficient because the function receives the address of each matrix instead of copying the whole structure.

### Why does the function return a pointer?

The result matrix is created inside the function.

If we want the result to still exist after the function ends, we create it on the heap and return its address.

## 7. First Rule: Dimensions Must Match

Two matrices can be added only if they have the same number of rows and columns.

### Valid example

```
S1 is 3 x 4  
S2 is 3 x 4
```

These can be added.

Why?

Because both matrices have:

```
3 rows and 4 columns
```

### Invalid example

```
S1 is 3 x 4  
S2 is 4 x 3
```

These cannot be added.

Why?

Because the positions do not match.

Matrix addition works position by position:

```
C[i][j] = A[i][j] + B[i][j]
```

So every position in the first matrix must have a matching position in the second matrix.

---

## 8. Dimension Check Code

Inside the `add()` function, we write:

```
if (S1->m != S2->m || S1->n != S2->n) {  
    return nullptr;  
}
```

This means:

If the row counts are different, addition is impossible.  
If the column counts are different, addition is impossible.  
Return nullptr to show that no result matrix was created.

Older C/C++ code may use `NULL`:

```
return NULL;
```

Modern C++ commonly uses:

```
return nullptr;
```

Both ideas mean:

There is no valid result pointer.

---

## 9. Why We Use -> in the Function

Inside the function, `S1` and `S2` are pointers.

So we use the arrow operator:

```
S1->m  
S1->n  
S1->num  
S1->e
```

and:

```
S2->m  
S2->n
```

```
S2->num  
S2->e
```

### Simple rule

If you have a pointer, use ->  
If you have a normal object, use .

Example:

```
S1->num
```

means:

Go to the Sparse object pointed to by S1, then access its num field.

---

## 10. Creating the Result Matrix

Inside `add()`, we need a new sparse matrix to store the result.

We call it `sum`.

```
struct Sparse *sum;
```

Then we create it dynamically:

```
sum = new Sparse;
```

Or in one line:

```
struct Sparse *sum = new Sparse;
```

---

## 11. Why the Result Matrix Must Be Created on the Heap

A common wrong idea is this:

```
struct Sparse sum;  
return &sum;
```

This is wrong because `sum` is a local variable.

A local variable exists only while the function is running.

When the function ends, that local variable is destroyed.

So returning its address is unsafe.

### Correct idea

```
struct Sparse *sum = new Sparse;
```

This creates `sum` on the heap.

Heap memory remains available after the function returns.

So it is safe to return:

```
return sum;
```

---

## 12. Copying the Matrix Dimensions

The result matrix has the same dimensions as `S1` and `S2`.

So we copy the row and column count from `S1`:

```
sum->m = S1->m;  
sum->n = S1->n;
```

We can copy from `S1` because the dimension check already proved that:

```
S1->m == S2->m  
S1->n == S2->n
```

So copying from either matrix would give the same result.

---

## 13. Result Array Size

At the beginning, we do not know the exact number of non-zero elements in the result.

So we allocate enough space for the worst case.

### Worst case

The worst case happens when no non-zero coordinates overlap.

Example:

```
S1 has non-zero elements at:  
(0, 0), (1, 2)
```

```
S2 has non-zero elements at:  
(0, 1), (2, 3)
```

No positions are the same.

So every non-zero element from both matrices appears in the result.

If:

```
S1->num = 2  
S2->num = 2
```

Then the result may need:

```
2 + 2 = 4 elements
```

So we allocate:

```
sum->e = new Element[S1->num + S2->num];
```

---

## 14. Why the Actual Result May Be Smaller

If both matrices have a non-zero value at the same position, those two values are added and stored as one result element.

Example:

```
S1 has (2, 5, 7)
S2 has (2, 5, 6)
```

Both values are at the same coordinate:

```
row = 2
column = 5
```

So we add the values:

```
7 + 6 = 13
```

The result stores only one element:

```
(2, 5, 13)
```

So even though two input elements were used, only one result element was created.

That is why the final result count may be smaller than:

```
S1->num + S2->num
```

---

## 15. Why We Set `sum->num = k` at the End

The variable `k` counts how many result elements we actually write.

Every time we store one element in `sum->e`, we increase `k`.

At the end:

```
k = actual number of non-zero elements in the result
```

So we must write:

```
sum->num = k;
```

Do not write:

```
sum->num = S1->num + S2->num;
```

That would only be the maximum capacity, not the actual number of result elements.

---

## 16. The Three Pointers: i, j, and k

Sparse matrix addition uses three index variables:

```
int i, j, k;
```

They start at zero:

```
i = j = k = 0;
```

### Meaning

| Variable | Meaning                         |
|----------|---------------------------------|
| i        | Current element in S1           |
| j        | Current element in S2           |
| k        | Current writing position in sum |

### Beginner mental model

Think of `i`, `j`, and `k` as three fingers.

```
i points to the current element in S1.  
j points to the current element in S2.  
k points to the place where the next result element will be written.
```

After copying or adding an element, the correct pointer moves forward.

---

## 17. Important Assumption: Row-Major Order

For this addition function to work correctly, the elements in both sparse matrices should be stored in row-major order.

### Row-major order means:

Sort by row first.  
If rows are equal, sort by column.

### Good order

(0, 1, 5)  
(0, 3, 9)  
(2, 0, 4)  
(3, 2, 8)

This is good because rows increase from top to bottom.

Inside the same row, columns increase from left to right.

### Bad order

(3, 2, 8)  
(0, 1, 5)  
(2, 0, 4)  
(0, 3, 9)

This is bad because the algorithm moves forward only.

It does not go backward and search again.

So if the sparse lists are not sorted, the result may be wrong.

---

## 18. Main Loop Condition

The main comparison loop is:

```
while (i < S1->num && j < S2->num)
```

This means:

```
Keep comparing while S1 still has elements and S2 still has elements.
```

The loop stops when either list ends.

After that, leftover elements from the other list must be copied.

---

## 19. Merge Logic Overview

At every step, compare:

```
S1->e[i]  
S2->e[j]
```

Each element has:

```
row, column, value
```

So we compare positions.

First compare rows.

If rows are equal, compare columns.

The smaller coordinate comes first in the result.

If the coordinates are the same, add the values.

---

## 20. Rule 1: S1 Row Is Smaller

Code:

```
if (S1->e[i].i < S2->e[j].i) {  
    sum->e[k++] = S1->e[i++];  
}
```

Meaning:

The current element in S1 comes before the current element in S2.  
So copy S1's element into the result.  
Then move i and k forward.

Example:

S1->e[i] = (1, 3, 7)  
S2->e[j] = (2, 1, 5)

Compare rows:

1 < 2

So copy:

(1, 3, 7)

Move:

i++  
k++

---

## 21. Rule 2: S2 Row Is Smaller

Code:

```
else if (S1->e[i].i > S2->e[j].i) {  
    sum->e[k++] = S2->e[j++];  
}
```

Meaning:

The current element in S2 comes before the current element in S1.  
So copy S2's element into the result.  
Then move j and k forward.

Example:

```
S1->e[i] = (2, 1, 7)
S2->e[j] = (1, 5, 4)
```

Compare rows:

```
2 > 1
```

So copy:

```
(1, 5, 4)
```

Move:

```
j++
k++
```

---

## 22. Rule 3: Rows Equal, S1 Column Is Smaller

If rows are equal, compare columns.

Code:

```
if (S1->e[i].j < S2->e[j].j) {
    sum->e[k++] = S1->e[i++];
}
```

Meaning:

```
Both elements are in the same row.
But S1's column comes first.
So copy S1's element into the result.
```

Example:

```
S1->e[i] = (2, 3, 7)
S2->e[j] = (2, 5, 4)
```

Rows are equal:

```
2 == 2
```

Compare columns:

```
3 < 5
```

So copy:

```
(2, 3, 7)
```

Move:

```
i++  
k++
```

---

## 23. Rule 4: Rows Equal, S2 Column Is Smaller

Code:

```
else if (S1->e[i].j > S2->e[j].j) {  
    sum->e[k++] = S2->e[j++];  
}
```

Meaning:

```
Both elements are in the same row.  
But S2's column comes first.  
So copy S2's element into the result.
```

Example:

```
S1->e[i] = (2, 5, 7)  
S2->e[j] = (2, 3, 4)
```

Rows are equal:

```
2 == 2
```

Compare columns:

```
5 > 3
```

So copy:

```
(2, 3, 4)
```

Move:

```
j++  
k++
```

---

## 24. Rule 5: Same Row and Same Column

This is the real addition case.

Code:

```
else {  
    sum->e[k].i = S1->e[i].i;  
    sum->e[k].j = S1->e[i].j;  
    sum->e[k].x = S1->e[i].x + S2->e[j].x;  
  
    i++;  
    j++;  
    k++;  
}
```

Meaning:

```
Both elements are at the same position.  
So add their values.  
Store one result element.  
Then move i, j, and k forward.
```

Example:

```
S1->e[i] = (2, 5, 7)
S2->e[j] = (2, 5, 6)
```

Same row:

```
2 == 2
```

Same column:

```
5 == 5
```

Add values:

```
7 + 6 = 13
```

Result:

```
(2, 5, 13)
```

Move:

```
i++
j++
k++
```

**Why move both i and j?**

Because we used one element from `S1` and one element from `S2`.

**Why move k?**

Because we wrote one result element.

---

## 25. Why We Copy Leftovers

The main loop runs only while both lists still have elements:

```
while (i < S1->num && j < S2->num)
```

If one list ends first, the loop stops.

But the other list may still have remaining elements.

Those remaining elements are called leftovers.

They must be copied into the result.

Why?

Because if an element is left in one matrix, it means there is no matching element in the other matrix.

So its value stays the same in the sum.

---

## 26. Copy Leftovers from S1

Code:

```
while (i < S1->num) {  
    sum->e[k++] = S1->e[i++];  
}
```

Meaning:

While S1 still has elements, copy them into sum.  
Move i and k forward each time.

---

## 27. Copy Leftovers from S2

Code:

```
while (j < S2->num) {  
    sum->e[k++] = S2->e[j++];  
}
```

Meaning:

While S2 still has elements, copy them into sum.  
Move j and k forward each time.

## 28. Full add() Function

Here is the full Lesson 5 function:

```
struct Sparse* add(struct Sparse *S1, struct Sparse *S2) {
    struct Sparse *sum;
    int i, j, k;

    if (S1->m != S2->m || S1->n != S2->n) {
        return nullptr;
    }

    sum = new Sparse;

    sum->m = S1->m;
    sum->n = S1->n;
    sum->e = new Element[S1->num + S2->num];

    i = j = k = 0;

    while (i < S1->num && j < S2->num) {
        if (S1->e[i].i < S2->e[j].i) {
            sum->e[k++] = S1->e[i++];
        }
        else if (S1->e[i].i > S2->e[j].i) {
            sum->e[k++] = S2->e[j++];
        }
        else {
            if (S1->e[i].j < S2->e[j].j) {
                sum->e[k++] = S1->e[i++];
            }
            else if (S1->e[i].j > S2->e[j].j) {
                sum->e[k++] = S2->e[j++];
            }
            else {
                sum->e[k].i = S1->e[i].i;
                sum->e[k].j = S1->e[i].j;
                sum->e[k].x = S1->e[i].x + S2->e[j].x;

                i++;
                j++;
            }
        }
    }

    return sum;
}
```

```

        k++;
    }
}

while (i < S1->num) {
    sum->e[k++] = S1->e[i++];
}

while (j < S2->num) {
    sum->e[k++] = S2->e[j++];
}

sum->num = k;

return sum;
}

```

## 29. Important Note About new and malloc

This Lesson 5 version uses C++ style memory allocation:

```

new Sparse
new Element[S1->num + S2->num]

```

In a pure C version, we would use `malloc` instead.

That comes more clearly in Lesson 6.

So for Lesson 5, focus on the addition logic and heap allocation idea.

## 30. Dry Run Example

Let the two sparse matrices be:

S1:

| Index | Row | Col | Value |
|-------|-----|-----|-------|
| 0     | 1   | 3   | 7     |
| 1     | 2   | 5   | 6     |

S2:

| Index | Row | Col | Value |
|-------|-----|-----|-------|
| 0     | 1   | 5   | 4     |
| 1     | 2   | 5   | 8     |

Assume both matrices have the same dimensions.

Initial values:

```
i = 0
j = 0
k = 0
```

---

## 31. Dry Run Step 1

Current elements:

```
S1->e[i] = S1->e[0] = (1, 3, 7)
S2->e[j] = S2->e[0] = (1, 5, 4)
```

Compare rows:

```
1 == 1
```

Rows are equal, so compare columns:

```
3 < 5
```

So **S1** comes first.

Copy **S1->e[0]** into **sum->e[0]**:

```
sum->e[0] = (1, 3, 7)
```

Move:

```
i = 1  
k = 1
```

j stays the same:

```
j = 0
```

Current result:

```
sum:  
Index  Row  Col  Value  
0      1    3    7
```

## 32. Dry Run Step 2

Current elements:

```
S1->e[i] = S1->e[1] = (2, 5, 6)  
S2->e[j] = S2->e[0] = (1, 5, 4)
```

Compare rows:

```
2 > 1
```

So S2 comes first.

Copy S2->e[0] into sum->e[1]:

```
sum->e[1] = (1, 5, 4)
```

Move:

```
j = 1  
k = 2
```

i stays the same:

`i = 1`

Current result:

sum:  

| Index | Row | Col | Value |
|-------|-----|-----|-------|
| 0     | 1   | 3   | 7     |
| 1     | 1   | 5   | 4     |

### 33. Dry Run Step 3

Current elements:

`S1->e[i] = S1->e[1] = (2, 5, 6)`  
`S2->e[j] = S2->e[1] = (2, 5, 8)`

Compare rows:

`2 == 2`

Compare columns:

`5 == 5`

Same row and same column.

So add the values:

`6 + 8 = 14`

Store one result element:

`sum->e[2] = (2, 5, 14)`

Move all three pointers:

```
i = 2  
j = 2  
k = 3
```

Now both lists are finished.

---

## 34. Final Result of Dry Run

The result is:

```
sum:  
Index  Row  Col  Value  
0      1    3    7  
1      1    5    4  
2      2    5   14
```

So:

```
sum->num = k = 3
```

Important:

```
S1->num = 2  
S2->num = 2  
S1->num + S2->num = 4
```

But the final result has only:

```
3 non-zero elements
```

Why?

Because one pair overlapped:

```
(2, 5, 6) + (2, 5, 8) = (2, 5, 14)
```

Two entries became one result entry.

---

## 35. Instructor Trace Cases from Lesson 5

Lesson 5 focuses on three important trace cases.

### Case 1: Rows Equal, Columns Different

Example:

```
S1 at (1, 3)
S2 at (1, 5)
```

Rows are equal:

```
1 == 1
```

Columns are different:

```
3 < 5
```

So copy **S1**:

```
Copy S1 element.
Move i and k.
```

---

### Case 2: Rows Different

Example:

```
S1 at (2, 1)
S2 at (1, 5)
```

Compare rows:

```
2 > 1
```

So **S2** row comes first.

Copy **S2**:

```
Copy S2 element.  
Move j and k.
```

---

### Case 3: Exact Match

Example:

```
S1 at (2, 5) with value 7  
S2 at (2, 5) with value 6
```

Same row and same column:

```
(2, 5) == (2, 5)
```

Add the values:

```
7 + 6 = 13
```

Store:

```
(2, 5, 13)
```

Move:

```
i++  
j++  
k++
```

---

## 36. What Happens If Values Add to Zero?

The basic Lesson 5 version usually stores the sum even if it becomes zero.

Example:

```
S1 has (1, 2, 5)
S2 has (1, 2, -5)
```

Adding gives:

```
5 + (-5) = 0
```

A perfect sparse matrix representation should not store zero values.

So a more careful version would skip this result.

But the basic lesson version focuses on the main merge logic and usually does not handle this edge case yet.

---

## 37. Complexity Analysis

### Time Complexity

The function scans each non-zero element at most once.

The pointer `i` only moves forward through `S1`.

The pointer `j` only moves forward through `S2`.

No element is searched repeatedly.

So the time complexity is:

```
O(S1->num + S2->num)
```

or simply:

```
O(num1 + num2)
```

where:

```
num1 = number of non-zero elements in S1
num2 = number of non-zero elements in S2
```

## Space Complexity

The result array is allocated with maximum possible size:

```
S1->num + S2->num
```

So the result space complexity is:

```
O(S1->num + S2->num)
```

or:

```
O(num1 + num2)
```

---

## 38. Why Sparse Addition Is Efficient

Normal matrix addition checks every position.

If a matrix has:

```
m rows  
n columns
```

then normal addition checks:

```
m * n positions
```

So normal matrix addition takes:

```
O(m * n)
```

Sparse matrix addition only checks stored non-zero elements.

So if the matrices are very sparse, sparse addition can be much faster.

Example:

```
Full matrix size = 1000 x 1000 = 1,000,000 positions  
S1 non-zero elements = 100  
S2 non-zero elements = 150
```

Normal addition checks:

```
1,000,000 positions
```

Sparse addition checks about:

```
100 + 150 = 250 stored elements
```

That is the advantage of sparse matrix addition.

---

## 39. Common Beginner Mistakes

### Mistake 1: Forgetting the dimension check

Wrong idea:

```
Add matrices even when their sizes are different.
```

Correct idea:

```
Only add when rows and columns match.
```

Code:

```
if (S1->m != S2->m || S1->n != S2->n) {  
    return nullptr;  
}
```

---

### Mistake 2: Allocating only the larger num

Wrong:

```
sum->e = new Element[max(S1->num, S2->num)];
```

This is not enough in the worst case.

Correct:

```
sum->e = new Element[S1->num + S2->num];
```

Why?

Because if no coordinates overlap, the result needs all elements from both matrices.

---

### Mistake 3: Setting sum->num incorrectly

Wrong:

```
sum->num = S1->num + S2->num;
```

Correct:

```
sum->num = k;
```

Why?

Because `k` is the actual number of result elements written.

---

### Mistake 4: Forgetting to copy leftovers

Wrong idea:

```
Stop after the main while loop.
```

Correct idea:

```
After the main loop, copy remaining elements from S1 or S2.
```

Code:

```
while (i < S1->num) {  
    sum->e[k++] = S1->e[i++];  
}  
  
while (j < S2->num) {  
    sum->e[k++] = S2->e[j++];  
}
```

---

### Mistake 5: Comparing columns before rows

Wrong order:

Compare columns first.

Correct order:

Compare rows first.  
If rows are equal, compare columns.

Why?

Because the elements are stored in row-major order.

---

### Mistake 6: Moving only one pointer when coordinates match

Wrong:

Same coordinate:  
add values  
i++  
k++

Correct:

Same coordinate:  
add values  
i++

```
j++  
k++
```

Why?

Because one element from `S1` and one element from `S2` were both used.

---

## Mistake 7: Returning the address of a local variable

Wrong:

```
struct Sparse sum;  
return &sum;
```

Correct:

```
struct Sparse *sum = new Sparse;  
return sum;
```

Why?

The local variable disappears when the function ends.

The heap object remains available after the function returns.

---

## Mistake 8: Not storing elements in row-major order

Wrong idea:

```
Elements can be entered randomly.
```

Correct idea:

```
For this merge algorithm, elements should be sorted by row first and column  
second.
```

---

## 40. Beginner Mental Model

Imagine two sorted lists of important locations.

Each location has:

```
row, column, value
```

You compare the first location from each list.

If one location comes earlier, copy it.

If both locations are the same, add their values.

Then move forward.

This is the same idea as merging two sorted lists.

That is why sparse matrix addition is efficient.

---

## 41. One-Line Memory Trick

```
Sparse matrix addition merges two sorted Coordinate Lists and adds values only  
when the coordinates match.
```

---

## 42. Important Code Symbols

| Symbol                      | Meaning                                    |
|-----------------------------|--------------------------------------------|
| <code>struct Element</code> | Stores one non-zero value                  |
| <code>struct Sparse</code>  | Stores the whole sparse matrix information |
| <code>m</code>              | Number of rows                             |
| <code>n</code>              | Number of columns                          |
| <code>num</code>            | Number of non-zero elements                |
| <code>e</code>              | Dynamic array of non-zero elements         |
| <code>S1</code>             | First sparse matrix pointer                |

| Symbol                        | Meaning                                    |
|-------------------------------|--------------------------------------------|
| <code>S2</code>               | Second sparse matrix pointer               |
| <code>sum</code>              | Result sparse matrix pointer               |
| <code>i</code>                | Current index in <code>S1</code>           |
| <code>j</code>                | Current index in <code>S2</code>           |
| <code>k</code>                | Current index in <code>sum</code>          |
| <code>new Sparse</code>       | Creates a result Sparse object on the heap |
| <code>new Element[...]</code> | Creates a dynamic array of Element records |
| <code>nullptr</code>          | Means no valid result pointer              |
| <code>-&gt;</code>            | Accesses a field through a pointer         |

## 43. Important Rules from Lesson 5

### Rule 1

Always check dimensions first.

```
if (S1->m != S2->m || S1->n != S2->n) {
    return nullptr;
}
```

### Rule 2

Create the result matrix on the heap.

```
sum = new Sparse;
```

### Rule 3

Allocate the maximum possible result size.

```
sum->e = new Element[S1->num + S2->num];
```

### Rule 4

Use three pointers.

```
i = j = k = 0;
```

### Rule 5

Compare rows first.

```
if (S1->e[i].i < S2->e[j].i)
```

### Rule 6

If rows are equal, compare columns.

```
if (S1->e[i].j < S2->e[j].j)
```

### Rule 7

If row and column both match, add the values.

```
sum->e[k].x = S1->e[i].x + S2->e[j].x;
```

### Rule 8

Copy leftovers after the main loop.

```
while (i < S1->num)
while (j < S2->num)
```

### Rule 9

Set the final result count using `k`.

```
sum->num = k;
```

### Rule 10

Return the result pointer.

```
return sum;
```

---

## 44. Practice Example

Trace this addition:

S1:

| Row | Col | Value |
|-----|-----|-------|
| 0   | 0   | 4     |
| 1   | 2   | 5     |
| 2   | 1   | 7     |

S2:

| Row | Col | Value |
|-----|-----|-------|
| 0   | 0   | 6     |
| 1   | 1   | 3     |
| 2   | 1   | 8     |

Initial values:

```
i = 0
j = 0
k = 0
```

### Step 1

```
S1[i] = (0, 0, 4)
S2[j] = (0, 0, 6)
```

Same row and same column.

Add:

```
4 + 6 = 10
```

Store:

```
sum[0] = (0, 0, 10)
```

Move:

```
i = 1  
j = 1  
k = 1
```

## Step 2

```
S1[i] = (1, 2, 5)  
S2[j] = (1, 1, 3)
```

Rows are equal.

Compare columns:

```
2 > 1
```

So copy `S2[j]`:

```
sum[1] = (1, 1, 3)
```

Move:

```
j = 2  
k = 2
```

## Step 3

```
S1[i] = (1, 2, 5)  
S2[j] = (2, 1, 8)
```

Compare rows:

```
1 < 2
```

So copy `S1[i]`:

```
sum[2] = (1, 2, 5)
```

Move:

```
i = 2  
k = 3
```

#### Step 4

```
S1[i] = (2, 1, 7)  
S2[j] = (2, 1, 8)
```

Same row and same column.

Add:

```
7 + 8 = 15
```

Store:

```
sum[3] = (2, 1, 15)
```

Move:

```
i = 3  
j = 3  
k = 4
```

Both lists are finished.

#### Final answer

```
sum:  
Row  Col  Value  
0     0     10  
1     1      3  
1     2      5  
2     1     15
```

So:

```
sum->num = 4
```

## 45. Self-Check Questions

### Question 1

When can two sparse matrices be added?

#### Answer

Only when they have the same number of rows and columns.

```
S1->m == S2->m  
S1->n == S2->n
```

---

### Question 2

Why do we allocate result space as `S1->num + S2->num`?

#### Answer

Because in the worst case, no coordinates overlap, so the result may need to store all non-zero elements from both matrices.

---

### Question 3

What does `i` do?

#### Answer

`i` points to the current element in `S1`.

---

### Question 4

What does `j` do?

#### Answer

`j` points to the current element in `S2`.

### Question 5

What does `k` do?

#### Answer

`k` points to the current writing position in the result matrix `sum`.

---

### Question 6

What happens if `S1->e[i]` and `S2->e[j]` have the same row and same column?

#### Answer

Their values are added and stored as one result element.

---

### Question 7

Why do we set `sum->num = k` at the end?

#### Answer

Because `k` is the actual number of result elements written.

---

### Question 8

What is the time complexity of the addition function?

#### Answer

$O(S1 \rightarrow \text{num} + S2 \rightarrow \text{num})$

or:

$O(\text{num1} + \text{num2})$

---

### Question 9

What is the worst-case space complexity?

## Answer

```
O(S1->num + S2->num)
```

or:

```
O(num1 + num2)
```

---

## Question 10

Why should elements be in row-major order?

## Answer

Because the algorithm moves forward through the arrays only.

Sorted order allows the function to compare and merge efficiently.

---

## 46. Lesson 5 Final Summary

Lesson 5 teaches how to turn sparse matrix addition logic into an actual program function.

The function is:

```
struct Sparse* add(struct Sparse *S1, struct Sparse *S2)
```

The main steps are:

1. Check if dimensions match.
2. If dimensions do not match, return nullptr.
3. Create a result Sparse object on the heap.
4. Copy m and n into the result.
5. Allocate result array of size S1->num + S2->num.
6. Start i = 0, j = 0, k = 0.
7. Compare rows first.
8. If rows are equal, compare columns.
9. Copy the smaller coordinate.
10. If coordinates match, add the values.
11. Copy leftovers after one list ends.

```
12. Set sum->num = k.  
13. Return sum.
```

The most important idea is:

```
Sparse matrix addition does not add every cell.  
It only merges the stored non-zero elements.
```

That is why the time complexity is:

```
O(S1->num + S2->num)
```

not:

```
O(m * n)
```

This lesson prepares you for Lesson 6, where the sparse matrix addition program is completed and tested in a full C-style implementation.